claranet®

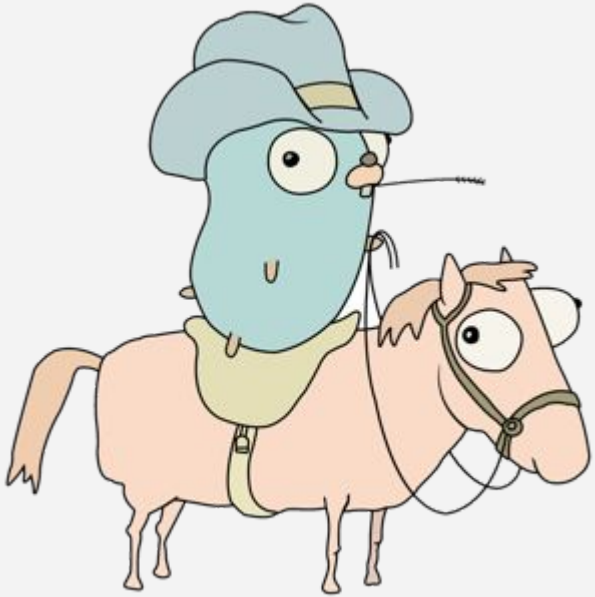
Formation Go 2026

Thèmes de l’atelier

Utilisation de WaitGroupes et mutexes

Les fonctions atomiques

Création de canaux



```go
func DoSomethingGreat(){
    fmt.Println("Hayyy !")
}
…
go DoSomethingGreat()
```

allez GO !!!

WaitGroup

```go
var wg sync.WaitGroup
…
func DoSomethingGreat(){
    fmt.Println("Hayyy !")
    wg.Done()
}
…
wg.Add(1)
go DoSomethingGreat()
wg.Wait()
```

mutex

```
1  var mutex = sync.Mutex{}
2  …
3  mutex.Lock()
4  i++
5  fmt.Println(i)
6  mutex.Unlock()
```

Fonctions atomiques

```
1  var i uint64
2  …
3  atomic.AddUint64(i, 1)
4  …
5  iFinal := atomic.LoadUint64(&i)
6
```

Canal non bufferisé

```go
msg := make(chan string)

go func(){
    msg <- "non buffered"
}()

fmt.Println(<-msg)
```

Canal bufferisé

```
1  msg := make(chan string, 1)
2
3  msg <- "buffered"
4
5  fmt.Println(<-msg)
6
```

Chan chan

```
1  msg := make(chan chan string)
2
3  msg <- "buffered"
4
5  fmt.Println(<-msg)
6
```

Select canal

```
1   msg := make(chan string)
2   stop := time.Tick(1 * time.Second)
3   go DoSomeThingGreat(msg)
4
5   select {
6   case <- msg :
7       fmt.Println("Yeee")
8   case <- stop :
9       fmt.Println("stop")
10      return
11  }
```

Select canal non bloquant

```go
msg := make(chan string)
go DoSomeThingGreat(msg)

select {
case <- msg :
    fmt.Println("Yeee")
default:
    fmt.Println("stop")
    return
}

```

Select récupération de close

```
1   msg := make(chan string, 1)
2   close(msg)
3   select {
4   case res, ok := <- msg :
5       if ok {
6           fmt.Println(res)
7       }else{
8           fmt.Println("chan is closed")
9       }
10  }
11
```

https://play.golang.org/p/Vg93cVxH_g

Pipeline

```go
func sq(in <-chan int) <-chan int {
	out := make(chan int)
	go func() {
		for n := range in {
			out <- n * n
		}
		close(out)
	}()
	return out
}
```

```go
func gen(nums ...int) <-chan int {
	out := make(chan int)
	go func() {
		for n := range nums {
			out <- n
		}
		close(out)
	}()
	return out
}
```

Pipeline

```go
func main() {
	// Set up the pipeline.
	c := gen(2, 3)
	out := sq(c)

	// Consume the output.
	fmt.Println(<-out) // 4
	fmt.Println(<-out) // 9
}
```

https://play.golang.org/p/-ISgne9dQK

Pipeline

```go
21 func main() {
22     for n := range sq(sq(gen(2, 3))) {
23         fmt.Println(n) // 16 then 81
24     }
25 }
26
27
28
29
30
```

https://play.golang.org/p/-ISgne9dQK

Paradigms

https://divan.github.io/posts/go_concurrency_visualize/

Worker pool

http://marcio.io/2015/07/handling-1-million-requests-per-minute-with-golang/